



MDMC

Master in Data Management
and Curation

AREA
SCIENCE PARK
■■■■



Scientific Programming Environment

Lecture II: Text manipulation

Jacopo Salvalaggio

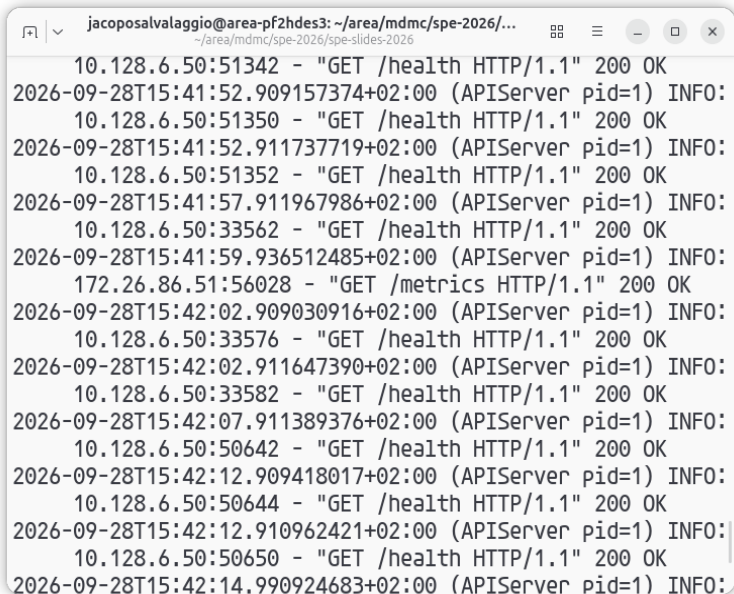
Special thanks to Gianfranco Gallizia



Image credit: vecteezy.com



Goals

A terminal window with a title bar showing the user 'jacoposalvalaggio@area-pf2hdes3' and the current directory '~/area/mdmc/spe-2026/...'. The terminal displays a series of log entries. Each entry consists of a timestamp, an IP address, a port, a method, a path, a status code, and an additional log message. The log messages include 'INFO:' and 'API Server pid=1'.

```
jacoposalvalaggio@area-pf2hdes3: ~/area/mdmc/spe-2026/...  
10.128.6.50:51342 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:41:52.909157374+02:00 (APIServer pid=1) INFO:  
10.128.6.50:51350 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:41:52.911737719+02:00 (APIServer pid=1) INFO:  
10.128.6.50:51352 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:41:57.911967986+02:00 (APIServer pid=1) INFO:  
10.128.6.50:33562 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:41:59.936512485+02:00 (APIServer pid=1) INFO:  
172.26.86.51:56028 - "GET /metrics HTTP/1.1" 200 OK  
2026-09-28T15:42:02.909030916+02:00 (APIServer pid=1) INFO:  
10.128.6.50:33576 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:42:02.911647390+02:00 (APIServer pid=1) INFO:  
10.128.6.50:33582 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:42:07.911389376+02:00 (APIServer pid=1) INFO:  
10.128.6.50:50642 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:42:12.909418017+02:00 (APIServer pid=1) INFO:  
10.128.6.50:50644 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:42:12.910962421+02:00 (APIServer pid=1) INFO:  
10.128.6.50:50650 - "GET /health HTTP/1.1" 200 OK  
2026-09-28T15:42:14.990924683+02:00 (APIServer pid=1) INFO:
```

Objectives of the lecture:

- * Learn more about **editing** and **parsing** text files with:
 - * **sed**
 - * **awk**

Why not just nano?

Text editors are great to **read and edit single files** of text, but sometimes we need to **perform edits to a group of files** in bulk or **restructure large files** to extract only some kind of information.

sed allows us to edit files from the command line, while **awk** is a powerful parsing tool (imagine it as overpowered cut)

sed

sed allows you to **edit files** from the **command line**. It can do many things and can be quite complicated! On a **basic level**, you can use it to:

- * Print a selected **range of lines**
- * Modify a line by appending or inserting text or deleting it
- * **Substitute strings** or characters

Of course, it supports **regular expressions**.

Examples of sed

Printing a set of lines from a file:

```
$ # Print line 6
$ sed -n '6p' FILENAME
$ # Print lines from 5 to 10
$ sed -n '5,10p' FILENAME
$ # Print lines 3, 6 and 9
$ sed -n '3p;6p;9p' FILENAME
```

Substituting strings in a file:

```
$ # Substitute all '-' with
 '+' on line 10
$ sed -i '10y/-/+' FILENAME
$ # Substitute all occurrences
of 'ciao' with 'hello'
$ sed -i 's/ciao/hello/g'
FILENAME
```

More on sed

sed can be used **on any string** with pipelines.

Example script to rename many files:

```
for filename in *; do
    new_filename=$(echo $filename | sed 's/2025/2026/g')
    if [ $new_filename != $filename ]; then
        mv $filename $new_filename
    fi
done
```



Always be **super careful** when doing this!

awk

awk is a powerful utility to **parse** text. It allows to:

- * Read contents of a file **line-by-line**
- * Manipulate the different **columns** of each line
- * Handle **logic** (e.g. if statements)



Image credit: Pearson

Comparison with cut

On a very basic level, **awk** and **cut** behave **similarly**.

Getting columns of a file:

```
$ cat simple_table.csv
2026,19,MDMC,SISSA and AREA
$ cut -d',' -f2,4 simple_table.csv
19,SISSA and AREA
$ awk -F',' '{ print $2 "," $4 }'
19,SISSA and AREA
```

However, **awk** is much **more powerful**.

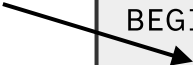
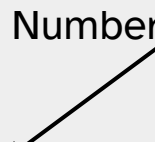
awk scripting

awk is a proper **programming language** that you can use to write scripts with **variables, lists, if** statements...

Columns in a file are called "fields" and lines are "records".

Field Separator

Example script counting words:

```
BEGIN {  
    FS = " "             
    word_counter = 0  
    the_counter = 0  
}  
{  
    for ( i = 1 ; i <= NF ; i++ ) {   
        word_counter++  
        if ( $i ~ /[tT]he/ ) {  
            the_counter++  
        }  
    }  
}  
END {  
    print("Word count:", word_counter)  
    print("'The' count:", the_counter)  
}
```

Another awk example

Reading a table with a header:

```
{
    if ( NR == 1 ) {
        for ( i = 1 ; i <= NF ; i++ ) {
            headers[i] = $i
        }
    } else {
        printf("%d: ", NR - 1)
        for ( i = 1 ; i < length(headers) ; i++ ) {
            printf("%s = %s, ", headers[i], $i)
        }
        printf("%s = %s.\n", headers[length(headers)],
$(length(headers)))
    }
}
```

Record Number

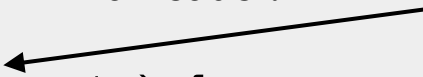




Image credit: Abel+Imray

Thank you for your attention :)

Image credits for background
rainbow gradient: Image Colorful
Rainbow Pattern by sgagnonou812 at
Wallpapers.com